

可以。我们先跳过实验，把 2026 这篇《Speech Tokenizers are Vulnerable: Transferable Semantic Attack and Robust Tokenizer》的理论部分读完。它和 2025 E2E-VGuard 的关系很清楚：

2025 E2E-VGuard → 保护语音克隆场景 → 同时扰乱音色和 ASR 文本

也就是说，2026 这篇不是简单“换一个 ASR 攻击方法”，而是提出：**现代语音大模型真正脆弱的位置可能是 speech tokenizer（语音 tokenizer）这个前端接口。**

生词

- **vulnerable**: 脆弱的，容易受到攻击的
- **transferable**: 可迁移的
- **semantic**: 语义的
- **tokenizer**: 分词器；这里指把连续语音变成离散 token 的模块
- **interface**: 接口，连接层
- **continuous**: 连续的
- **discrete**: 离散的
- **representation**: 表示，表征
- **fragile**: 脆弱的
- **perturbation**: 扰动
- **degradation**: 下降，退化
- **downstream**: 下游的
- **ensemble**: 集成
- **drift**: 漂移，偏移
- **amplification**: 放大
- **collapse**: 崩塌
- **fidelity**: 保真度
- **distillation**: 蒸馏
- **reconstruction**: 重建
- **robustness**: 鲁棒性
- **holistic**: 整体的
- **bottleneck**: 瓶颈
- **post-training**: 后训练
- **surrogate**: 替代的，代理的
- **quantizer**: 量化器
- **front end**: 前端模块
- **compromise**: 损害，破坏
- **propagation**: 传播
- **underexplored**: 研究不足的

1. Abstract + Introduction：这篇论文到底换了哪个攻击视角？

这篇论文开头说，**speech tokenizer（语音 tokenizer）** 是 continuous speech signals（连续语音信号）和 discrete representations（离散表示）之间的 critical interface（关键接口）。也就是说，现代 ASR 和 **LALM — Large Audio-Language Model — 大音频语言模型** 并不是直接把原始波形交给 LLM 理解，而是通常先经过 speech tokenizer，把连续波形变成 token 或中间表示，再送给后续模型。论文认为这个接口很关键，因为 tokenizer 的鲁棒性会直接影响整个 semantic pipeline（语义流水线）的可靠性。

这里先把“为什么 tokenizer 重要”拆开：

waveform → speech tokenizer → speech tokens / semantic representations → ASR or LALM → text / understanding / reasoning

如果 tokenizer 在第一步就把语音内容编码错了，后面的 ASR 或 LALM 就是在错误 token 上继续推理。后面的模型再强，也可能是在“错误输入”上做强推理。

这就是论文和 2025 E2E-VGuard 的第一个差异。2025 那篇主要是：

攻击 ASR 输出 → 让转录文本错

而 2026 这篇更前移：

攻击 tokenizer / semantic encoder → 让语义表示本身漂移 → ASR 和 LALM 都被影响

作者提出 **T-SemAttack — Transferable Semantic Attack — 可迁移语义攻击**。它不是直接针对某个 ASR 的解码输出，而是联合扰动多个 semantic encoders（语义编码器）的 representation spaces（表示空间），让语音的 semantic content（语义内容）被破坏，同时通过 psychoacoustic model（心理声学模型）保持较好的感知质量。论文还说，这种小扰动会沿着 tokenizer pipeline 被逐步放大，导致 semantic collapse（语义崩塌），例如在 S3 tokenizer 上造成 99.7% 的 token change（token 改变）。

刚刚这个思路本质上对应：

- **T-SemAttack — Transferable Semantic Attack — 可迁移语义攻击。**
 - **Transferable** 表示它不是只对一个模型有效，而希望迁移到未知 ASR / LALM。
 - **Semantic** 表示攻击目标不是单纯噪声、音色或最后文本，而是语音的语义表示。
 - **Attack** 表示通过小扰动使模型理解错误。

注意，这篇论文的防御部分不是“检测假音频”，而是提出 **ROSETok — RObust SpEech Tokenizer — 鲁棒语音 tokenizer**。它结合 noisy training（噪声训练）和 robust semantic distillation（鲁棒语义蒸馏），目标是让 tokenizer 在噪声输入下仍能高保真重建，并保持下游任务鲁棒性。

因此，这篇论文的主线是双结构：

Attack → T-SemAttack → 证明 speech tokenizer 语义脆弱

2. Introduction 的核心逻辑：为什么不是继续攻击整个 ASR？

Introduction 进一步说明，当前 LLM-based speech architectures（基于大语言模型的语音架构）通常依赖 speech tokenizer 把连续信号映射成离散 token，再送进 LLM 建模。因此 speech tokenizer 实际上连接了 continuous acoustic space（连续声学空间）和 discrete semantic space（离散语义空间）。论文指出，以前很多研究关注的是 model-level holistic robustness（整体模型鲁棒性），但相对忽略 tokenizer 作为 signal input module（信号输入模块）的安全问题。

这段的逻辑可以写成：

已有攻击/防御 → 多关注整个 ASR / LALM 输出

为什么这重要？因为攻击整个 ASR 输出通常依赖某个模型的 decoding objective（解码目标），例如 CTC loss。这样有两个问题：第一，优化成本高；第二，它容易把前端语音表示的作用遮住。论文明确说，很多旧方法攻击 entire ASR pipeline（整个 ASR 流水线），常依赖 CTC 这类 surrogate decoding objectives（代理解码目标），但 tokenizer-level transfer attacks（tokenizer 层面的迁移攻击）和它的传播机制仍然研究不足。

所以这篇论文的切入点是：

不直接问：怎样让 ASR 输出错？

而是问：

怎样让 speech tokenizer 的语义表示先错？

这比 2025 更“底层”。2025 是破坏 E2E 克隆链路中的 ASR 和音色特征；2026 是直接破坏现代语音大模型的 tokenizer 前端。

3. Related Work: speech tokenizer 是什么？

论文在 Related Work 里给了一个很关键的定义：speech tokenizer 通常由 **encoder (编码器)** 和 **quantizer / decoder (量化器 / 解码器)** 组成。encoder 先把 speech signals (语音信号) 编码成 representations (表示)，quantizer 再把这些表示离散化成 tokens，例如 VQ-based 或 FSQ-based tokens。

这里要先把 speech tokenizer 拆开：

waveform $\rightarrow$ encoder $\rightarrow$ continuous representation $\rightarrow$ quantizer $\rightarrow$ discrete token

- **Encoder — 编码器。**
 - 负责从原始音频里提取连续表示。
- **Quantizer — 量化器。**
 - 负责把连续表示映射到有限个离散 code / token。
- **VQ — Vector Quantization — 向量量化。**
 - 用一个 codebook (码本) 里的向量代表连续空间中的某片区域。
- **FSQ — Finite Scalar Quantization — 有限标量量化。**
 - 用有限的标量级别对表示进行离散化。

你之前说“codebook 定义了人类发音所有可能方式”这个理解可以保留一半，但需要修正：**codebook 不是严格定义了所有人类发音方式，而是在模型训练分布下，用有限离散向量近似语音表示空间。**它不是语言学意义上的完整音素表，而是模型为了压缩、建模和生成而学到的离散表示集合。

论文强调，speech tokenizer 是 modern speech foundation models (现代语音基础模型) 的 front end (前端)，用于 ASR、TTS、LALM 等任务。因为它桥接 continuous acoustic space 和 discrete semantic space，所以这个组件的安全漏洞会直接损害整个 speech foundation model 的可靠性。

这句话很重要：

tokenizer 错 $\rightarrow$ token 错 $\rightarrow$ LLM 输入错 $\rightarrow$ 语义理解错

4. Threat Model: 攻击者和防御者分别知道什么？

Section 3.1 是威胁模型。作者考虑的是 transfer-based attack scenario (基于迁移的攻击场景)。攻击者可以访问 input speech waveform (输入语音波形)，并拥有一组 differentiable surrogate semantic encoders (可微的代理语义编码器)，但不能访问目标系统内部的 gradients (梯度)、parameters (参数) 或 tokenizer details (tokenizer 细节)。在噪声仍可接受的约束下，攻击者希望生成 adversarial examples (对抗样本)，让语音系统产生错误 transcription (转录)。同时，这些对抗样本还要能抵抗 denoising (降噪) 和 data transformations (数据变换)。论文还说明，这个攻击是 offline setting (离线场景)，所以实时要求较低。

这里的攻击者能力是：

能访问原始音频

这就不是 white-box (白盒) 攻击，而更接近 black-box transfer (黑盒迁移) 攻击。攻击者不需要知道目标 ASR / LALM 的完整结构，只要能在代理语义编码器上构造出能迁移的语义漂移。

防御者假设也比较强：论文假设 defender (防御者) 可以检测 embedded noise (嵌入噪声)，并使用现有工具或策略尽量恢复原始音频。这说明作者不是在假设一个很弱的防御方，而是承认对方可能降噪、变换、修复音频。

5. Problem Definition: 总目标函数是什么？

Section 3.2 把攻击形式化。给定原始音频：

$$x \in [-1, 1]^T$$

其中 T 是音频长度。攻击目标是构造：

$$x_{\text{adv}} = x + \delta$$

其中 δ 是小的 adversarial perturbation (对抗扰动)。扰动要满足两个条件：第一，**Semantic Disruption (语义破坏)**，即 x_{adv} 应该在 semantic representation spaces (语义表示空间) 里偏离 x ，导致 ASR 和 LALM 输出不同；第二，**Perceptual Constraint (感知约束)**，即扰动对人类听者来说尽量可接受，同时满足幅度约束和心理声学约束。

总目标函数是：

$$\min_{\delta} L = \lambda_{\text{sem}} L_{\text{sem}}(x, x_{\text{adv}}) + \lambda_{\text{psy}} L_{\text{psy}}(x, \delta) + \lambda_2 \|\delta\|_2$$

约束是：

$$\|\delta\|_{\infty} \leq \epsilon, \quad x_{\text{adv}} = x + \delta \in [-1, 1]^T$$

其中 L_{sem} 衡量语义表示偏离， L_{psy} 表示心理声学掩蔽， $\|\delta\|_2$ 控制扰动大小。论文设置 $\lambda_{\text{sem}} = 100$ 、 $\lambda_{\text{psy}} = 10^{-3}$ 、 $\lambda_2 = 0.1$ 。

这三个 loss 的职责是：

$$L_{\text{sem}} \rightarrow \text{让语义表示漂移}$$

这和 2025 E2E-VGuard 的结构很像，但攻击目标不同：

$$2025: L_{\text{asr}} + L_{\text{fea}} + L_{\text{psy}}$$

差别在于：2026 不再显式把 ASR 输出文本作为核心目标，而是把 tokenizer / semantic encoder 的 representation drift 作为核心目标。

6. Tokenizer-centric Semantic Attack：为什么攻击 tokenizer encoder?

Section 3.3 的核心句是：不同于直接攻击 ASR decoding outputs (解码输出) 的方法，T-SemAttack 以 speech tokenizer 为中心。原因是 speech tokenizer 是语音系统中最早执行 representation learning (表示学习) 和 mapping (映射) 的模块；一旦 tokenizer 被攻击，后续 LLM 或 decoder 很难恢复原始语义信息。

这个判断很关键：

$$\text{前端 tokenizer 语义错} \rightarrow \text{后端模型只能处理错误表示} \rightarrow \text{恢复难度很大}$$

为了提升 cross-model transferability (跨模型迁移性)，作者同时优化多个 surrogate encoders (代理编码器)。具体包括四条分支：S3 Tokenizer encoder，直接模拟 speech tokenizer 的语义编码过程；HuBERT，提供 self-supervised learning (自监督学习) 得到的深层语义表示；Whisper encoder，提供强 ASR 预训练学到的鲁棒语义表示；MFCC，作为额外的 low-level attack target (低层攻击目标)。

所以四个代理分支可以理解为：

$$\text{S3 Tokenizer Encoder} \rightarrow \text{最贴近 tokenizer 目标}$$

设代理编码器集合为：

$$F = \{f_1, \dots, f_K\}$$

每个编码器对输入 x 输出：

$$f_k(x)$$

语义损失写成：

$$L_{\text{sem}} = \sum_{k=1}^K \text{CosSim}(f_k(x), f_k(x_{\text{adv}}))$$

论文说最小化这个式子，会同时驱动多个 semantic spaces (语义空间) 发生 shift (偏移)，从而提升对未知 ASR 和 LALM 的迁移性。

这里要注意一个公式方向：如果最小化 cosine similarity，就意味着让 clean representation 和 adversarial representation 的方向尽量不相似。

$$\text{CosSim 高} \rightarrow \text{语义表示接近}$$

所以：

$$\min L_{\text{sem}} \rightarrow \text{最大化语义漂移}$$

为什么它能迁移？论文给出的机制解释是：多个 semantic encoders 虽然训练目标不同，但共享相似的 Mel-input bottlenecks（Mel 输入瓶颈）和 semantic structures（语义结构）。因此，在代理模型中扰乱共享语义表示，往往也会扰乱未知目标模型的对应表示。

这句话可以翻译成更直观的话：

不同语音模型 $\rightarrow$ 前端通常都要看 Mel / 声学结构 $\rightarrow$ 高层都要编码语义 $\rightarrow$ 攻击这些共同结构更容易迁移

7. Psychoacoustic Model: 这篇的 L_{psy} 和 2025 有什么关系？

T-SemAttack 也使用 psychoacoustic constraint（心理声学约束）。作者说，以前可迁移 ASR 攻击常有感知质量低的问题，所以他们用心理声学约束让对抗扰动对人类保持可接受。给定原始音频 x ，先基于其 short-time spectrum（短时频谱）计算 masking threshold（掩蔽阈值） $\Theta(x)$ ，作为扰动能量的上界参考。设 $PSD(\delta)$ 是扰动在 time-frequency domain（时频域）中的 power spectral density（功率谱密度），则心理声学损失为：

$$L_{\text{psy}} = \sum_{t,f} \max(0, PSD_{t,f}(\delta) - \Theta_{t,f}(x))$$

它只在扰动能量超过某个时频单元的掩蔽阈值时惩罚。也就是说：

$$PSD_{t,f}(\delta) \leq \Theta_{t,f}(x) \rightarrow \text{不惩罚}$$

这和你刚刚问 2025 里的 $\theta_x(f)$ 是同一类思想，但 2026 写得更清楚：阈值依赖具体样本 x ，并且是 time-frequency unit（时频单元）级别的，不是写死的全局常数。

8. Optimization Process: 它怎么更新扰动？

优化时，T-SemAttack 提取多个 representation，然后根据总 loss 更新 adversarial audio。论文采用 **PGD-style projected update**（**PGD 风格投影更新**），每一步后把扰动投影回 ℓ_∞ ball（ ℓ_∞ 范数球）中：

$$\delta \leftarrow \text{Proj}_{\|\delta\|_\infty \leq \epsilon} (\delta - \alpha \cdot \text{sign}(\nabla_\delta L))$$

- **PGD — Projected Gradient Descent — 投影梯度下降。**

- 先沿梯度方向更新扰动；
- 再把扰动拉回允许范围内；
- 防止扰动超过 ϵ 。

这里的 ℓ_∞ 约束意思是：每个采样点的最大改变量不能超过 ϵ 。它控制的是逐点最大扰动，而 $\|\delta\|_2$ 控制整体能量。

论文还说，实际中结合 SGD optimizer（随机梯度下降优化器）和 adaptive plateau learning-rate scheduler（自适应平台期学习率调度器），当 loss stagnates（停滞）时自动降低学习率，提高优化稳定性；整个过程运行 500 iterations（迭代）。

9. Why T-SemAttack Works: 为什么小扰动能把 tokenizer 打崩？

Section 4 是理论分析里最值得读的部分。它解释 T-SemAttack 为什么有效，而不是只给实验结果。

第一是 **Token Fragility — Token 脆弱性**。论文说，T-SemAttack 会破坏 tokenizers 的 discretization results（离散化结果），在 LibriTTS 上造成多个 tokenizer 极高的 token change rates，例如 S3 Tokenizer 99.702%、MOSS-Audio-Tokenizer 96.666%、DAC 96.418%、StableToken 94.454%。语义攻击破坏了 speech 到 discrete space（离散空间）的映射，因此增加了 LLM 建模难度。

这里的核心是：

小波形扰动 $\rightarrow$ 连续表示漂移 $\rightarrow$ 量化边界跨越 $\rightarrow$ token 大量改变

为什么 token 很容易变？因为 quantizer 本质上有边界。如果原始表示刚好靠近某个 codebook cell（码本区域）的边界，一点扰动就可能把它推到另一个 code。连续空间里的小位移，在离散 token 空间里可能表现为 token ID 的突然变化。

第二是 **Perturbation Amplification — 扰动放大**。论文发现，在 S3 Tokenizer 上，小扰动会沿着：

Waveform ($1\times$) $\rightarrow$ Mel-spectrogram ($5.01\times$) $\rightarrow$ Encoder ($9.87\times$)

逐步放大。也就是说，扰动不是进入 tokenizer 后保持原样，而是在 Mel 变换和 encoder 表示过程中被放大。

第三是 **Layer-wise Similarity — 层级相似度**。论文比较 clean 和 adversarial representation 在不同层的 cosine similarity。HuBERT 从输入嵌入层的 0.881 下降到最终层的 0.079；Whisper 早期层较稳定，但最终层也下降到 0.854；S3 Tokenizer 的高层表示甚至变成负相关 -0.291 。这说明 shallow acoustic representations（浅层声学表示）相对更稳，但扰动接近 semantic layers（语义层）时会被 Transformer 架构累积和放大，轻量 tokenizer 尤其脆弱。

这条机制可以写成：

浅层 $\rightarrow$ 更多声学局部特征 $\rightarrow$ 相对稳定

第四是 **LALM Attention Disruption — 大音频语言模型注意力破坏**。论文分析 Qwen2-Audio，发现高层 Transformer 中 target text 到 audio tokens 的 attention ratio（注意力比例）平均下降约 15%–25%，减少的注意力主要转移到 target text 本身。也就是说，攻击后的 audio tokens 让模型不再可靠依赖音频上下文，跨模态建模被削弱，理解错误进一步放大。

这里的因果链是：

tokenizer 输出错误 token $\rightarrow$ audio token 可信度下降 $\rightarrow$ LALM 减少对音频的注意力 $\rightarrow$ 更多依赖文本上下文 $\rightarrow$ 语音理解错误

10. ROSETok：防御为什么要回到 tokenizer 层？

Section 5 是防御理论。作者基于前面分析提出一个重要判断：有些 speech tokenizers 不具备 semantic robustness（语义鲁棒性），容易受到对抗攻击。作者认为只在 LLM 层做防御更耗时、成本更高，所以要回到问题核心：提高 tokenizer against noise（对噪声的稳定性）。分析显示扰动会在 Mel-spectrogram domain（梅尔频谱域）被放大，深层语义信息会漂移，因此 tokenizer 内部应满足两个要求：noisy training（噪声训练）和 semantic preservation（语义保持）。

- **ROSETok — RObust SpEech Tokenizer — 鲁棒语音 tokenizer。**
 - 目标不是检测攻击；
 - 目标是让 tokenizer 对噪声输入仍能重建高质量音频，并保持语义稳定。

模型架构上，ROSETok 基于 DAC 设计 convolutional architecture（卷积架构）。encoder 接收 waveform 输入，将其压缩 320 倍，产生 50 Hz representation；quantizer 包含 9 层 **RVQ — Residual Vector Quantization — 残差向量量化**；decoder 镜像 encoder。

这里的 RVQ 可以这样理解：

第 1 层量化 $\rightarrow$ 捕捉主要信息

训练上，ROSETok 使用 noisy audio（带噪音频）作为输入，用 clean audio（干净音频）作为 reconstruction 和 semantic signals 的监督。训练输入由 20% clean audio 和 80% noisy audio 组成。

这条训练逻辑是：

noisy input $\rightarrow$ encoder $\rightarrow$ quantizer $\rightarrow$ decoder $\rightarrow$ clean reconstruction

它希望模型学会：

有噪声输入 $\rightarrow$ 仍输出干净、高质量语音

但只做 reconstruction 不够。因为前面的攻击分析显示，真正危险的是 deep semantic information（深层语义信息）丢失。因此 ROSETok 加入 **Semantic Preservation Module — 语义保持模块**。它使用 pretrained Semantic Teacher（预训练语义教师）Whisper 来监督模型 encoder 的表示，使 noisy inputs 的表示和 clean audio 的表示对齐，从而保持语义信息，并提高下游任务鲁棒性。

这里对应：

- **Semantic Teacher — 语义教师。**
 - 用一个已有强模型提供语义监督。

- **Semantic Distillation — 语义蒸馏。**
 - 让学生 tokenizer 的表示接近教师模型的语义表示。
- **Semantic Preservation — 语义保持。**
 - 目标不是 token ID 完全不变，而是深层语义表示不要漂移。

ROSETok 的总训练目标是：

$$L_{\text{tok}} = \lambda_{\text{distill}} L_{\text{distill}} + \lambda_{\text{recon}} L_{\text{recon}} + \lambda_{\text{cb}} L_{\text{cb}} + \lambda_{\text{commit}} L_{\text{commit}} + L_{\text{adv}} + L_{\text{fm}}$$

其中 L_{distill} 是 semantic distillation loss（语义蒸馏损失）， L_{recon} 是 reconstruction loss（重建损失）， L_{cb} 是 codebook loss（码本损失）， L_{commit} 是 commitment loss（承诺损失）， L_{adv} 是 adversarial loss（对抗生成损失）， L_{fm} 是 feature matching loss（特征匹配损失）。论文设置 $\lambda_{\text{distill}} = 5.0$ 、 $\lambda_{\text{recon}} = 15$ 、 $\lambda_{\text{cb}} = 1.0$ 、 $\lambda_{\text{commit}} = 0.25$ ， L_{adv} 和 L_{fm} 权重为 1。

这几个 loss 可以按功能分成三类：

$$L_{\text{recon}} + L_{\text{adv}} + L_{\text{fm}} \rightarrow \text{保证重建音频质量}$$

11. 这篇理论部分的一句话总结

2026 这篇论文的理论是：现代 ASR / LALM 的安全瓶颈不只在最终解码器，而在 speech tokenizer 这个把连续语音变成离散语义 token 的前端；T-SemAttack 通过联合扰动多个代理语义编码器，使小波形扰动在 Mel、encoder、quantizer 链条中被放大，造成 token 变化、语义漂移和 LALM 注意力转移；ROSETok 则试图用 noisy-clean reconstruction 和 semantic distillation，让 tokenizer 在噪声下仍保持重建质量和深层语义稳定。

下一步更适合继续精读 T-SemAttack 的三个核心问题： L_{sem} 到底如何定量算、为什么 Mel-input bottleneck 会带来迁移性、ROSETok 的 L_{distill} 、 L_{cb} 、 L_{commit} 分别在训练中约束什么

下面专注 T-SemAttack 里的 L_{sem} 和量化表示，不讨论 ROSETok。你指出的问题是对的：论文说攻击 speech tokenizer 并诱导 error tokens，但它的优化目标 L_{sem} 实际并不是直接优化离散 token，而是优化 tokenizer encoder 前后的连续语义表示；token 变化更多是后续量化边界被跨越后的结果。

生词

- **quantization（量化）**：把连续向量映射成有限离散编号或离散向量。
- **codebook（码本）**：量化器中保存的一组可选向量。
- **boundary（边界）**：连续表示空间中不同 code 对应区域的分界线。
- **surrogate（代理）**：不能直接攻击目标模型时，用来近似目标的替代模型。
- **proxy（代理目标）**：不直接优化最终目标，而优化一个可微、可操作的替代目标。
- **differentiable（可微的）**：可以反向传播梯度。
- **non-differentiable（不可微的）**：不能直接稳定反向传播梯度。
- **representation（表征）**：模型中间层输出的向量表示。
- **semantic drift（语义漂移）**：语义表示从原本方向偏移。
- **token collapse（token 崩塌）**：离散 token 序列大面积改变，导致语义失真。

1. 先明确：speech tokenizer 的量化链条是什么

论文中对 speech tokenizer 的基本定义是：

$$\text{speech signal} \rightarrow \text{encoder} \rightarrow \text{continuous representation} \rightarrow \text{quantizer} \rightarrow \text{discrete tokens}$$

论文明确说，speech tokenizer 通常由 **encoder（编码器）** 和 **quantizer / decoder（量化器 / 解码器）** 组成；encoder 把语音信号编码成 representations，quantizer 再把这些 representations 离散化成 specific tokens，例如 **VQ — Vector Quantization — 向量量化** 或 **FSQ — Finite Scalar Quantization — 有限标量量化** tokens。

所以这里至少有三层对象：

$$x \rightarrow z \rightarrow c$$

其中：

$$x \rightarrow \text{原始 waveform (波形)}$$

如果是 VQ，典型形式是：

$$\mathcal{C} = \{e_1, e_2, \dots, e_K\}$$

这里：

- z_t : 第 t 个时间帧的连续向量。
- e_j : codebook 里的第 j 个向量。
- c_t : 第 t 帧被分配到的离散 code ID。
- q_t : 用 codebook 向量替代后的量化表示。

也就是说，量化不是“理解语义”本身，而是：

$$\text{连续语义/声学向量} \rightarrow \text{最近的 codebook 向量} \rightarrow \text{离散 token ID}$$

这一步有一个重要性质：**它是分段常数映射**。连续向量 z_t 在同一个 code 区域内部小幅移动时，token ID 可能不变；但一旦跨过边界，token ID 会突然改变。

2. L_{sem} 实际优化的不是 c ，而是 z

论文给出的语义损失是：

$$L_{\text{sem}} = \sum_{k=1}^K \text{CosSim}(f_k(x), f_k(x_{\text{adv}}))$$

其中 f_k 是第 k 个 surrogate semantic encoder（代理语义编码器），输出的是 representation，而不是离散 token。论文原文也说：对输入 x ，encoder f_k 输出 representations $f_k(x)$ ，然后用 frame-wise averaged cosine similarity（逐帧平均余弦相似度）定义 semantic loss；最小化它会推动多个 semantic spaces 发生 shift。

所以严格来说：

$$L_{\text{sem}} \neq \text{token-level loss}$$

而是：

$$L_{\text{sem}} = \text{continuous representation loss}$$

也就是它优化的是：

$$f_k(x) \quad \text{和} \quad f_k(x_{\text{adv}})$$

之间的方向差异，而不是直接优化：

$$Q(f_k(x)) \quad \text{和} \quad Q(f_k(x_{\text{adv}}))$$

之间的 token 差异。

这里是论文没有完全落实到位的地方：**它的叙事强调 speech tokenizer 和 error tokens，但主优化目标没有显式写成 token-level objective，而是使用 encoder-level semantic representation objective 作为代理目标。**

3. 代码也证实：T-SemAttack 用的是 encoder 输出，不是离散 token

开源代码更清楚。README 说明 T-SemAttack 的攻击脚本使用 S3 Tokenizer、HuBERT、Whisper 和 MFCC 的 semantic representation loss，并结合 psychoacoustic masking loss、 L_2 regularization 和 projected gradient updates。

具体到 `attack_semantic.py`：

- S3 分支里，代码加载 S3 tokenizer，然后取的是 `self.speech_tokenizer.encoder`，也就是 tokenizer 的 encoder 部分，而不是完整的 quantizer 输出。
- `_extract_s3` 里，代码先算 `log_mel_spectrogram`，然后过 `self.encoder_s3(mel, mel_len)`，返回的是 `hidden.transpose(2, 1)`。这就是 encoder hidden representation，不是 codebook indices。
- `get_vectors` 返回四类连续特征：S3 hidden、HuBERT hidden、Whisper hidden、MFCC。
- `compute_feature_loss` 对这些连续向量逐项计算 `CosineSimilarity(...).mean()`，四个分支权重都是 1.0。
- 优化时，`loss_feature = self.compute_feature_loss(wave_vectors, adv_vectors)`，总 loss 是 `loss_feature * 100 + loss_psy * 1e-5 + loss_l2 * 0.1`。

所以代码层面可以写成：

$$L_{\text{sem}} = \text{CosSim}(z_{\text{clean}}^{S3}, z_{\text{adv}}^{S3}) + \text{CosSim}(z_{\text{clean}}^{\text{HuBERT}}, z_{\text{adv}}^{\text{HuBERT}}) + \text{CosSim}(z_{\text{clean}}^{\text{Whisper}}, z_{\text{adv}}^{\text{Whisper}}) + \text{CosSim}(z_{\text{clean}}^{\text{MFCC}}, z_{\text{adv}}^{\text{MFCC}})$$

其中：

$$z_{\text{clean}}^{S3} = E_{S3}(x)$$

而不是：

$$L = \text{distance}(c_{\text{clean}}, c_{\text{adv}})$$

这说明 T-SemAttack 的核心不是“直接让 token ID 不同”，而是“让量化前的连续语义表示漂移，进而使后续量化 token 发生变化”。

4. 那为什么连续 L_{sem} 能导致离散 token 大面积改变？

关键在于 **量化边界**。

假设 codebook 中有三个向量：

$$e_1, e_2, e_3$$

某一帧 clean representation 是：

$$z_t = E(x)_t$$

量化器会选最近的 code：

$$c_t = \arg \min_j \|z_t - e_j\|_2$$

如果 z_t 原来离 e_1 最近，那么：

$$Q(z_t) = 1$$

现在 T-SemAttack 通过 L_{sem} 让 adversarial representation 变成：

$$z'_t = E(x_{\text{adv}})_t$$

只要 z'_t 穿过了 e_1 和 e_2 的 Voronoi boundary (Voronoi 边界)，量化结果就会从：

$$Q(z_t) = 1$$

变成：

$$Q(z'_t) = 2$$

注意，这里连续空间里的变化可以不大，但离散 token 会突然跳变。这个现象可以概括为：

continuous drift $\rightarrow$ boundary crossing $\rightarrow$ discrete token change

所以，T-SemAttack 的实际攻击链不是：

直接优化 token ID

而是：

优化连续语义表示漂移 $\rightarrow$ 跨过 quantizer 边界 $\rightarrow$ token ID 改变 $\rightarrow$ LALM / ASR 输入语义错

论文中的机制分析也支持这一点。它说 T-SemAttack 会破坏 tokenizers 的 discretization results（离散化结果），并在多个 tokenizer 上造成极高 token change rates，例如 S3 Tokenizer 99.702%、MOSS-Audio-Tokenizer 96.666%、DAC 96.418%、StableToken 94.454%。论文解释为：语义攻击破坏了 speech 到 discrete space 的映射，从而增加 LLM 建模难度。

5. 论文里的“99.7% token change”是分析指标，不是训练目标

这一点要非常明确。

论文在 Appendix G.1 里分析 S3 Tokenizer 的内部链条：

Waveform $\rightarrow$ Mel-spectrogram $\rightarrow$ Encoder $\rightarrow$ VQ Tokens

它说为了分析 T-SemAttack 的机制，计算 clean audio 和 adversarial audio 在 tokenizer 不同内部阶段的 representation differences，并以 S3 Tokenizer 为例。

它定义了 final discrete VQ tokens 的 **TER — Token Error Rate — token 错误率**：

$$TER = 1 - \frac{\sum_{n=1}^N \mathbf{1}[c_n(x) = c_n(x_{\text{adv}})]}{N}$$

其中 $c_n(\cdot)$ 表示第 n 个 discrete token， N 表示 token 总数。

这说明：

$TER \rightarrow$ 评估 token 改变了多少

但它不是优化时的 loss。优化时用的是：

$L_{\text{sem}} \rightarrow$ 连续 representation cosine similarity

然后分析时再看：

$TER \rightarrow$ 最终 token 是否被改变

所以更严谨的说法是：

T-SemAttack 并没有把量化 token 本身纳入可微训练目标，而是用 encoder-level semantic drift 作为 proxy，然后用 TER 证明这种 drift 最终会被量化成大量 token 错误。

6. 为什么不直接优化 token-level loss?

这篇论文没有详细解释这个工程选择，但从量化结构看，原因大概率有三个。

第一，tokenization 里的 arg min 或 nearest-code selection 通常不可微：

$$c_t = \arg \min_j \|z_t - e_j\|_2$$

$\arg \min$ 输出的是离散 index，梯度不稳定或不可直接传播。你如果直接写：

$$\mathcal{L}_{token} = \mathbf{1}[c_t(x) = c_t(x_{adv})]$$

它几乎不可微，不能像普通神经网络 loss 那样直接优化。

第二，不同 speech tokenizer 的 token space 不统一。S3、DAC、MOSS、StableToken 可能有不同 codebook 数量、帧率、量化层数和 token 定义。直接优化某一个 tokenizer 的 token ID，迁移性可能变差。论文强调 T-SemAttack 要通过多个 surrogate semantic encoders 共享 Mel-input bottlenecks 和 semantic structures 来迁移到 unseen target models。

第三，token 变化不一定等价于语义变化。论文在讨论 ROSETok 时有一句值得借用来理解这个问题：token changes do not always imply semantic degradation (token 改变不一定意味着语义退化)；关键机制不是简单改变 token ID，而是驱动 encoder representations 发生 semantic drift，使下游模型难以恢复原始内容。虽然这里出现在 ROSETok 问答里，但它反过来也解释了 T-SemAttack 为什么更关注 encoder-level semantic drift。

因此，直接优化 token-level mismatch 可能会产生一个问题：

token ID 变了 \centernot $\Rightarrow$ 语义一定错了

而优化深层语义表示漂移更接近攻击目标：

semantic representation drift $\Rightarrow$ 下游 ASR/LALM 更难恢复

7. “量化表示”在这里有两种含义，论文有点混用

这里需要拆清楚。论文里的“quantized / discrete representation”至少可以指两层：

7.1 第一层：连续 encoder representation

$$z = E(x)$$

这是还没有量化的连续向量。T-SemAttack 的 L_{sem} 主要作用在这一层。

代码中 S3 分支返回的是：

$$z^{S3} = E_{S3}(\text{Mel}(x))$$

HuBERT、Whisper、MFCC 分支也是连续特征。

这一层的优点是可微、可优化、跨模型可对齐。

7.2 第二层：离散 token representation

$$c = Q(z)$$

这是经过 quantizer 之后的离散 code ID。论文用 TER 来评估这一层是否改变。

这一层的优点是直接对应 LLM 输入或 tokenizer 输出，但缺点是不可微、不同 tokenizer 不统一。

所以论文的真实逻辑应该写成：

$L_{sem} \rightarrow \text{continuous semantic representation drift} \rightarrow \text{quantizer boundary crossing} \rightarrow \text{discrete token error}$

而不是简单写成：

$L_{sem} \rightarrow \text{direct token attack}$

这就是你说的“没有落实到位”的核心：**论文标题和叙事强调 speech tokenizer / tokens，但公式和代码实际落在 encoder representation 层；token 层主要用于结果分析，而不是用于损失函数设计。**

8. 为什么小波形扰动会在量化前被放大？

论文给出的证据是 progressive amplification chain（渐进放大链）：

$$\text{Waveform (1}\times\text{)} \rightarrow \text{Mel-spectrogram (5.01}\times\text{)} \rightarrow \text{Encoder (9.87}\times\text{)} \rightarrow \text{VQ Tokens}$$

在 Appendix G.1 中，论文报告：waveform 阶段平均 RD 只有 0.154；Mel-spectrogram 阶段升到 0.771，对应 5.01× 放大；encoder output 阶段升到 1.520，对应 9.87× 放大；最终 VQ token level 的 TER 达到 99.7%。

这里的机制可以拆成三步：

8.1 Waveform 到 Mel：频域变换放大局部差异

$$x \rightarrow \text{STFT} \rightarrow \text{Mel filterbank} \rightarrow \log$$

Mel 特征不是简单复制 waveform。它会做窗口化、频率聚合、对数压缩。对抗扰动如果被优化到某些敏感频带，经过 Mel 变换后，差异可能比 waveform 层更明显。

代码中 Whisper 分支也手写了 differentiable log-Mel：先 `torch.stft`，再取 magnitude squared，再乘 Mel filters，最后取 `log10`。这说明攻击是可以通过 Mel 前端反向传播的。

8.2 Mel 到 Encoder：深层语义表示放大方向偏移

encoder 不只是线性变换。Transformer / CNN / self-supervised encoder 会逐层抽象，把局部声学差异整合成高层语义表示。论文发现 HuBERT 的 clean/adversarial cosine similarity 从输入 embedding 的 0.881 降到最终层 0.079，S3 Tokenizer 高层甚至负相关 -0.291。

这意味着：

$$\text{浅层差异小} \rightarrow \text{深层语义方向差异大}$$

8.3 Encoder 到 VQ Tokens：连续偏移被离散化成 token 跳变

当 encoder output z 发生方向偏移后，quantizer 会把它重新分配到 codebook 中最近的 code。连续空间的方向改变一旦跨过大量 codebook 边界，token sequence 就会大面积改变。

这就是：

$$\text{RD/CD 增大} \rightarrow \text{cross codebook boundary} \rightarrow \text{TER 接近 100\%}$$

9. 更严谨的 L_{sem} 应该如何写？

论文公式写得比较简化。考虑不同 encoder 的时间长度和维度可能不同，更贴近代码和理论的写法应该是：

$$L_{\text{sem}} = \sum_{k=1}^K w_k \cdot \frac{1}{T_k} \sum_{t=1}^{T_k} \cos(f_k(x)_t, f_k(x_{\text{adv}})_t)$$

其中：

$$f_k(x) \in \mathbb{R}^{T_k \times d_k}$$

代码中四个分支权重都是：

$$w_k = 1$$

并且遇到 shape 不一致时，会截断到相同的 `min_seq` 和 `min_len` 后再计算 cosine similarity。

如果要把“量化层”显式纳入理论，应该额外定义：

$$c_k(x) = Q_k(f_k(x))$$

然后定义：

$$TER_k = 1 - \frac{1}{N_k} \sum_{n=1}^{N_k} \mathbf{1}[c_{k,n}(x) = c_{k,n}(x_{\text{adv}})]$$

但由于 Q_k 常常不可微，所以它更适合作为 evaluation metric，而不是直接训练 loss。这正是论文实际做法。

10. 这部分的关键判断

你的质疑可以总结成一个很准确的判断：

T-SemAttack 名义上攻击 speech tokenizer 的离散 token，但可优化的 L_{sem} 实际落在 tokenizer encoder 和其他代理语义编码器的连续表示空间；离散 token 的改变不是直接优化出来的，而是连续语义漂移经过 quantizer 边界后产生的结果。

更完整的理论链条应该写成：

$$x \rightarrow \text{Mel front-end} \rightarrow z = E(x) \rightarrow c = Q(z) \rightarrow \text{ASR} / \text{LALM}$$

T-SemAttack 优化的是：

$$\min_{\delta} \sum_k \text{CosSim}(f_k(x), f_k(x + \delta)) + \lambda_{\text{psy}} L_{\text{psy}} + \lambda_2 \|\delta\|_2$$

它希望造成：

$$f_k(x + \delta) \approx f_k(x)$$

然后间接导致：

$$Q(E(x + \delta)) \neq Q(E(x))$$

最终表现为：

$$TER \approx 99.7\%$$

所以这篇论文的理论贡献不是“提出了一个新的可微 token-level loss”，而是提出了一个更实际的代理路线：

不要直接优化不可微 token $\rightarrow$ 优化多个可微 semantic encoder 的连续表示漂移 $\rightarrow$ 利用 tokenizer 量化边界把连续漂移放大成离散 token 崩塌

这条路线是合理的，但表述上确实容易让人误以为它直接在 token 层做攻击。实际不是。

25 论文的代码实现基本服务于论文公式，但很多细节在论文中被抽象掉了；26 论文的 T-SemAttack 叙事更容易产生误解，因为它强调 speech tokenizer / error tokens，但实际优化主要落在连续 encoder representation，而不是离散 token 本身。

一、2025 E2E-VGuard：论文与代码的主要差异

1. $M(\cdot)$ ：论文写得像“声学特征提取器”，代码具体落成 MFCC

论文中 $M(\cdot)$ 写作 **MFCC extractor**，并说它用于补充 acoustic features，从而扰动 speaker identity、articulation 和 prosodic patterns。论文公式中只写：

$$CS(M(x), M(x'))$$

但没有展开 M 的具体参数。论文也说明实验中六个 encoders / extractors 包括 VITS、GSV、MFCC、WavLM、CosyVoice CAM++、StyleTTS2 style encoder。

代码层面则更具体： M 不是泛泛的 Mel spectrogram，而是 **torchaudio 的 MFCC 变换**，之前我们看代码时对应的是 `torchaudio.transforms.MFCC(sample_rate=16000, n_mfcc=40)`。所以这里不是严格“论文和代码矛盾”，而是：

$$\text{论文：} M = \text{MFCC extractor}$$

更准确的表述是：**论文省略了 MFCC 的维度、采样率、帧级处理方式；代码把它具体化为 40 维 MFCC 特征。**

2. x_t : 论文说“最不相似目标说话人”，代码用多特征总 cosine similarity 最小来选

论文说 targeted timbre protection 会构造 speaker database，然后为每条受保护音频选择 feature space 中最不相似的 speaker，而不是随机选择异性别 speaker。公式中只写：

$$L_{\text{fea}}(x') = - \sum_{i=1}^k CS(E_i(x_t), E_i(x')) - CS(M(x_t), M(x'))$$

但没有展开“最不相似”具体怎么算。

代码实际把“最不相似”落成一个工程搜索过程：

$$x_t = \arg \min_{s \in \mathcal{D}} \text{compute_feature_loss}(\text{embs}(x), \text{embs}(s))$$

也就是遍历 speaker database，对每个候选 speaker 提取多路特征，再计算总 cosine similarity，选总相似度最低的。这里的特征不是单一 speaker embedding，而是与 L_{fea} 对应的多 encoder / MFCC 特征集合。

所以这里的差异是：

论文：最不相似 speaker by feature distance

论文说的是概念，代码给出了具体判据。

3. L_{asr} : 论文写成通用 $F(\text{ASR}(x'), y_t)$ ，代码按 ASR 类型分支实现

论文把发音损失写成：

$$L_{\text{asr}}(x') = F(\text{ASR}(x'), y_t)$$

并举例说，Wav2Vec2 可以用 CTC — Connectionist Temporal Classification — 连接时序分类 loss。

代码实际不是所有 ASR 都统一 CTC。代码导入了 `Wav2Vec2Processor`、`Wav2Vec2ForCTC`，并初始化了 `torch.nn.CTCLoss(blank=0)` 和 `CrossEntropyLoss(ignore_index=-100)`，说明实际实现里至少存在 CTC 和 cross entropy 两类路径。

实际应理解为：

Wav2Vec2 / Conformer / CitriNet → CTC loss

所以差异是：

论文： F 是 ASR objective 的抽象占位

这不是错误，而是论文为了统一表达，把不同 ASR loss 包进了 F 。

4. L_{psy} : 论文主公式较简化，代码按时频掩蔽阈值实现

论文主文中写：

$$L_{\text{psy}}(x') = \frac{1}{F} \sum_{f=1}^F \max(0, p_{x'-x}(f) - \theta_x(f))$$

看起来只按频率 f 求和。论文同时说 θ_x 是由 original audio x 得到的 masking threshold。

代码里的 masker 更具体：它对每条原始音频做 STFT，计算 PSD，再计算 Bark scale、absolute threshold of hearing、局部 masker，最后得到全局 masking threshold。代码中 `_compute_masking_threshold` 明确以输入样本 x 为参数，并通过 STFT、PSD、Bark、quiet threshold 等步骤计算阈值。

所以实际实现更接近：

$$L_{\text{psy}} = \sum_{t,f} \max(0, PSD_{t,f}(\delta) - \Theta_x(t, f))$$

差异是：

论文：频率维度简写

这里代码比论文公式更具体、更合理。

5. L_2 ：论文主目标函数没显式放进 Eq. (1)，但算法和代码会合并进感知项

论文 Eq. (1) 写的是：

$$L(x') = L_{\text{asr}}(x') + \alpha L_{\text{fea}}(x') + \beta L_{\text{psy}}(x')$$

后面又单独引入：

$$L_2(x') = \|x' - x\|_2$$

在 Appendix B 的算法里， C_3 实际写成：

$$C_3 = L_{\text{psy}} + \|x' - x\|_2$$

也就是说，主文公式没有把 L_2 明确写进总目标，但算法中把它和 psychoacoustic loss 合并处理。

所以这里是一个论文表达层级不一致：

主文 Eq. (1)：只写 L_{psy}

6. PGD 更新：论文伪代码更像 sign 更新，代码存在更多优化工程细节

论文 Appendix B 把扰动更新写成 PGD-style：

$$\delta = \text{Clamp}(-\text{sign}(\nabla_x C), -\epsilon, \epsilon)$$

然后：

$$x' = x + \delta$$

代码中存在学习率、plateau 调度等工程变量，这些在论文主文中没有展开。代码文件里可以看到 `lr_optim`、`plateau_length`、`max_lr`、`min_lr` 等优化相关设置。

这里我不把它说成“矛盾”，更准确是：

论文伪代码：突出 PGD 思想

二、2026 T-SemAttack：论文与代码的主要差异，不含 ROSETok

1. 最大差异：论文强调 token collapse，但代码优化的是连续 representation loss

论文摘要和 Introduction 强调 speech tokenizer，把连续信号变成 discrete tokens，并说 T-SemAttack 会诱导 error tokens、造成 token-level collapse，甚至在 S3 tokenizer 上造成 99.7% token change。

但是方法部分其实已经暴露了真实落点：论文说 T-SemAttack focuses on the encoder in the speech tokenizer，并且 L_{sem} 是多个 surrogate semantic encoders 的 frame-wise averaged cosine similarity。代码 README 也明确写：攻击脚本优化的是来自 S3 Tokenizer、HuBERT、Whisper、MFCC 的 **semantic representation loss**，再加 psychoacoustic masking loss、 L_2 正则和 projected gradient updates。

所以真实链条是：

代码实际优化 → 连续 encoder representations

不是：

直接优化 discrete token ID

更严谨地说：

$$L_{\text{sem}} = \sum_k \text{CosSim}(f_k(x), f_k(x_{\text{adv}}))$$

其中 $f_k(x)$ 是连续表示，不是 $Q(f_k(x))$ 的离散 token。

所以这是 26 论文最重要的“叙事 vs 实现”差异：

论文标题/摘要：speech tokenizer / error tokens

2. “攻击 speech tokenizer”在代码里具体是攻击 S3 Tokenizer encoder，不是完整 tokenizer

论文说 speech tokenizer 通常由 encoder 和 quantizer/decoder 组成，encoder 编码 speech signals，quantizer 将其离散化成 tokens。但 T-SemAttack 的攻击部分选择的是 speech tokenizer 的 **encoder**，不是完整 tokenizer 的 quantizer 输出。

代码 README 也写得很直接：semantic attack pipeline 使用的 surrogate encoders 包括 **S3 Tokenizer encoder from CosyVoice v1**、HuBERT-Large、Whisper-Large-v3 encoder 和 MFCC acoustic features。

所以差异是：

论文对象：speech tokenizer 作为整体安全瓶颈

也就是说，代码没有直接在 quantizer / codebook index 上做反向优化。

3. 论文说 tokenizer-level attack，但实际是 multi-surrogate representation attack

论文把 T-SemAttack 定义为 tokenizer-centric semantic attack，但公式里的 surrogate branches 包括 S3 Tokenizer encoder、HuBERT、Whisper encoder、MFCC。代码 README 也列出同样四类 surrogate encoders。

这意味着实际攻击不是单一 tokenizer attack，而是：

S3 tokenizer encoder + HuBERT + Whisper + MFCC

共同定义的 representation-space attack。

所以更准确的命名应是：

tokenizer-centric $\neq$ tokenizer-only

它是以 tokenizer 为核心动机，但实现上是多代理语义编码器集成攻击。

4. 论文中的 token change rate 是分析结果，不是训练目标

论文 Section 4 报告 token fragility，称 T-SemAttack 在多个 tokenizer 上造成高 token change rate，例如 S3 Tokenizer 99.702%、MOSS 96.666%、DAC 96.418%、StableToken 94.454%。它还说扰动沿着：

waveform ($1\times$) $\rightarrow$ Mel-spectrogram ($5.01\times$) $\rightarrow$ encoder ($9.87\times$)

被放大。

但代码运行部分说明优化 adversarial waveform 使用的是 semantic representation loss、psychoacoustic masking loss、 L_2 regularization 和 projected gradient updates。

所以应区分：

训练/优化目标: representation cosine similarity

论文有时容易让读者感觉“直接攻击 token”，但实际是：

先让连续表示漂移 → 再跨越量化边界 → 最终 token 改变

5. 超参数位置：论文写公式值，代码示例要求直接改脚本

论文写：

$$\lambda_{\text{sem}} = 100, \quad \lambda_{\text{psy}} = 10^{-3}, \quad \lambda_2 = 0.1$$

并说使用 500 iterations。

代码 README 则说主要超参数目前直接在 `attack_semantic.py` 里编辑，包括 `audio_path`、`output_path`、`epsilon`、`max_items`、`device`、`verbose`；默认示例使用 `epsilon=8/255` 和 `max_items=500`。

这里差异不是数学目标不同，而是工程接口不完整：

论文：给出统一公式和固定超参数

也就是说，代码更像 research prototype，而不是完整命令行工具。

三、最值得记住的总表

论文	论文说法	代码/实际更准确说法	性质
2025	M 是 MFCC extractor	具体是 16kHz、40 维 MFCC 特征	论文省略实现参数
2025	x_t 是最不相似目标 speaker	在 speaker database 中用多路特征 cosine similarity 最低者	论文概念化，代码具体化
2025	F 是 ASR objective，如 CTC	Wav2Vec2/Conformer/CitriNet 用 CTC，Whisper 用 cross entropy	论文统一抽象，代码分支实现
2025	L_{psy} 按频率 f 写	代码按样本相关的 STFT/PSD/Bark/ATH 时频阈值算	论文公式简化
2025	Eq. (1) 未显式写 L_2	算法/实现把 L_2 合入感知约束项	主文与算法表达不完全一致
2026	攻击 speech tokenizer / error tokens	实际优化 S3/HuBERT/Whisper/MFCC 连续表示	最大叙事差异
2026	token collapse 是核心现象	token change 是后验分析指标，不是直接 loss	容易误读
2026	tokenizer-level attack	实际是 tokenizer encoder + 多 surrogate encoder attack	tokenizer-centric，不是 tokenizer-only
2026	给出固定公式超参数	代码是脚本式 prototype，参数直接改 <code>attack_semantic.py</code>	工程接口不完整

一句话总结

25 论文的问题主要是“公式抽象、代码补细节”：MFCC 参数、目标 speaker 选择、ASR loss 分支、心理声学阈值实现都在代码里更具体；
26 论文的问题更偏“叙事重心和实际优化对象不完全一致”：论文强调 speech tokenizer 和 token collapse，但代码真正优化的是多个可微 semantic encoder 的连续表示漂移，离散 token 崩塌是后续量化边界被跨越后的结果，而不是直接优化目标。